

Splay-деревья

Задача

Хотим множество, с которым можно делать insert, delete и find за какое-то хорошее амортизированное время.

Что такое splay

Будем хранить элементы в вершинах двоичного дерева, которое будем перестраивать по ходу выполнения операций. Перестроение будет осуществляться за счет операции $\text{splay}(x)$, где x — какая-то вершина дерева. Выполнение $\text{splay}(x)$ сдвигает x в корень дерева с помощью поворотов вокруг ребер.

Поворот вокруг ребра выглядит вот так:

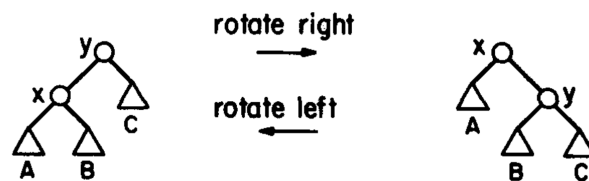


Рис. 1: Поворот относительно ребра xy

$\text{splay}(x)$ состоит из операций трех типов:

1. (zig) если $p(x)$ — корень дерева, поворачиваем дерево относительно ребра $p(x)x$.
2. (zig-zig) если $p(x)$ — не корень дерева, при этом $p(x)$ и x одновременно левые дети или одновременно правые дети, поворачиваемся относительно ребра $p(p(x))p(x)$, а затем относительно ребра $p(x)x$.
3. (zig-zag) если $p(x)$ — не корень дерева, а $p(x)$ — левый ребенок, а x — правый (или наоборот), то сначала поворачиваемся относительно ребра $p(x)x$ (y — x меняется предок), а затем снова поворачиваемся относительно ребра $p(x)x$.

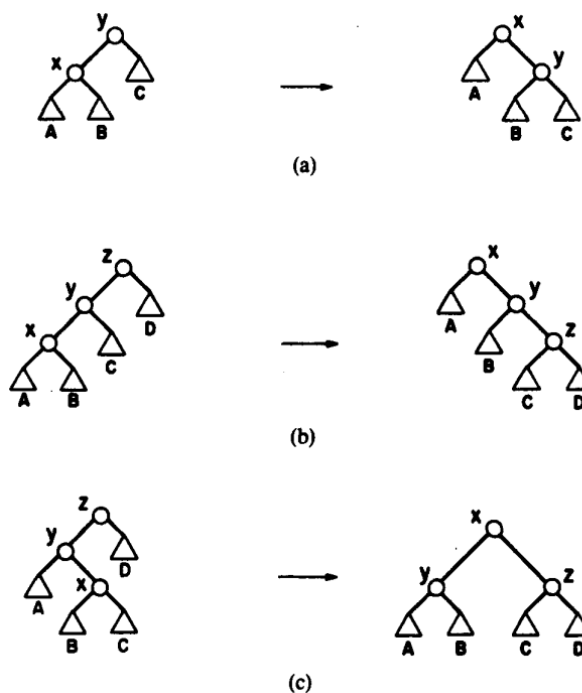


Рис. 2: Разные шаги $\text{splay}(x)$. а) zig: поворот xy б) zig-zig: поворот yz , затем поворот xy в) zig-zag: поворот xy , затем поворот xz

В процессе выполнения операции $\text{splay}(x)$ мы выполняем нужные перестроения (zig-zig или zig-zag) до тех пор, пока x не станет корнем или не придется сделать один zig, после которого x станет корнем.

$\text{splay}(x)$ работает за $\Theta(d)$, где d — глубина вершины x .

Понятно, что мы хотим, чтобы выполнение $\text{splay}(x)$ приближало дерево к сбалансированному виду. Если посмотреть на картинку, то видно, что это будет достигаться за счет операции zig-zag: после нее вершинки перераспределяются чуть более равномерно между поддеревьями. Докажем, что это происходит при помощи функции потенциала.

Пусть у каждого элемента i будет свой вес $w(i)$. Для каждой вершинки x определим ее size $s(x)$, равный сумме $w(i)$ по всем вершинам в поддереве x . Также определим ранг вершины x : $r(x) = \log s(x)$. Потенциалом дерева назовем сумму всех рангов вершин в дереве.

Лемма 1. Амортизированное время выполнения $\text{splay}(x)$ в дереве с корнем t не больше чем $3(r(t) - r(x)) + 1 = O(\log(s(t)/s(x)))$

Доказательство. Предположим, что хотя бы один поворот был выполнен (иначе это очевидное утверждение). Рассмотрим один шаг $\text{splay}(x)$. Пусть функции s , s' и r , r' являются функциями size и ранга до и после этого шага. Пусть $y = p(x)$ и $z = p(y)$ (если родитель вообще существует) до выполнения шага.

1. (zig) Выполнен один поворот. Тогда амортизированное время шага это

$$1 + r'(x) + r'(y) - r(x) - r(y) \leq 1 + r'(x) - r(x) \leq 1 + 3(r'(x) - r(x))$$

так как только x и y меняют ранг, $r(y) \geq r'(y)$, $r'(x) \geq r(x)$

2. (zig-zig) Выполнено два поворота. Тогда амортизированное время шага это

$$\begin{aligned} 2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) &= 2 + r'(y) + r'(z) - r(x) - r(y) \\ &\leq 2 + r'(x) + r'(z) - 2r(x) \end{aligned}$$

так как ранг меняют только x , y , z ; $r'(x) = r(z)$ (видно на картинке); $r'(x) \geq r'(y)$ и $r(y) \geq r(x)$. Заметим, что из неравенства Йенсена $2 \leq 2r'(x) - r(x) - r'(z)$, откуда $2 + r'(x) + r'(z) - 2r(x) \leq 3(r'(x) - r(x))$, что и требовалось.

3. (zig-zag) Почти то же самое:

$$2 + r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \leq 2 + r'(y) + r'(z) - 2r(x)$$

Из неравенства Йенсена $2 \leq 2r'(x) - r'(y) - r'(z)$, откуда $2 + r'(y) + r'(z) - 2r(x) \leq 2(r'(x) - r(x)) \leq 3(r'(x) - r(x))$, что и требовалось.

Осталось телескопически сложить оценки по всем шагам и получить, что амортизированное время не больше чем $3(r'(x) - r(x)) + 1 = 3(r(t) - r(x)) + 1$, где r и r' это функции ранга до и после $\text{splay}(x)$. $\square$

Заметим, что здесь у нас есть свобода выбора весов w . Действительно, если веса не меняются в процессе работы с деревом, то изменение потенциала дерева равно $\sum \log(W/w(i))$, где $W = \sum w(i)$. Тогда поподбирав веса можно получить всякие прикольные оценки на время.

Пусть задана последовательность из m доступов к элементам на дереве из n вершин.

Теорема 1. Общее время на выполнение будет $O((n + m) \log(n) + m)$

Доказательство. Пусть $w(i) = \frac{1}{n}$, $W = 1$. Затраченное амортизированное время равно $\sum_{i=1}^m 3(r_i(t_i) - r_i(x_i)) + 1 \leq m + 3m \log(n)$, а максимальное изменение потенциала равно $n \log n$, откуда следует оценка на время. $\square$

Пусть $q(i)$ — количество запросов на элемент i .

Теорема 2. Если каждый элемент запросили хотя бы раз, то общее время на выполнение будет

$$O(m + \sum_{i=1}^n q(i) \log\left(\frac{m}{q(i)}\right))$$

Доказательство. Надо определить каждому элементу вес $q(i)/m$ и повторить то же рассуждение. $\square$

Понятно, что веса можно придумывать еще миллионом разных способов и получать другие интересные оценки.

Как работает splay-дерево и за сколько

Имплементируем стандартные операции для нашей задачи

$\text{access}(i, t)$: Если элемент i есть в дереве t , вернуть указатель на него, иначе вернуть нулевой указатель.

$\text{insert}(i, t)$: Вставить элемент i в дерево t , предполагаем, что его там еще нет.

$\text{delete}(i, t)$: Удалить элемент i из дерева t , предполагаем, что он там есть.

$\text{join}(t_1, t_2)$: Мердж двух деревьев, считаем, что второе мажорирует первое.

$\text{split}(i, t)$: создать два дерева t_1, t_2 , где первое содержит все элементы t меньше либо равные i , а второе — все большие i .

Чтобы выполнить $\text{access}(i, t)$ мы сначала ищем элемент i сверху вниз по дереву. Если мы нашли i — делаем splay с ним, иначе делаем splay по последней посещенной вершиной в поиске.

Чтобы выполнить join , сначала делаем access с максимальным элементом в t_1 , назовем его i . После этого дерево будет выглядеть как i в корне, а все оставшееся дерево будет висеть на его левом ребенке. Достаточно сделать корень t_2 правым ребенком i .

Чтобы выполнить $\text{split}(i, t)$, сначала делаем $\text{access}(i)$, после чего удаляем корень и возвращаем два поддерева висевших на нем.

Чтобы выполнить $\text{delete}(i, t)$, находим i в вершине x . Пусть ее родитель это y . Делаем join с двумя поддеревьями x , после чего заменяем x как ребенка y результатом join . После этого делаем $\text{splay}(y)$.

Чтобы выполнить $\text{insert}(i, t)$, находим место, где должен быть i , заменяем нулевой указатель на i , и затем делаем $\text{splay}(i)$.

Чтобы провести анализ амортизированного времени этой конструкции, будем считать что у нас есть набор деревьев, каждый элемент находится ровно в одном дереве в каждый момент времени. Потенциалом набора деревьев считаем сумму потенциалов деревьев плюс сумму логарифмов весов элементов, которые еще не были добавлены ни в какие деревья.

Введем также полезные обозначения: Для любого элемента i в дереве t , $i-(i+)$ будут обозначать элемент предшествующий(следующий после) i в дереве t . Если они не определены, введем также $w(i-) = w(i+) = \infty$.

Лемма 2. Амортизированное время выполнения операций со splay-деревом имеют следующие верхние оценки

$$\text{access}(i, t): 3 \log \left(\frac{W}{w(i)} \right) + 1$$

$$\text{join}(t_1, t_2): 3 \log \left(\frac{W}{w(i)} \right) + 1 \text{ (здесь } i \text{ — наибольший элемент в } t_1)$$

$$\text{split}(i, t): 3 \log \left(\frac{W}{w(i)} \right) + 1$$

$$\text{insert}(i, t): 3 \log \left(\frac{W - w(i)}{\min\{w(i-), w(i+)\}} \right) + \log \left(\frac{W}{w(i)} \right) + 1$$

$$\text{delete}(i, t): 3 \log \left(\frac{W}{w(i)} \right) + 3 \log \left(\frac{W - w(i)}{w(i-)} \right) + 1$$

В access и split есть случаи, когда i не в t , тогда знаменатель в дроби надо заменить на $\min\{w(i-), w(i+)\}$.

Доказательство. Пусть s — size-функция до выполнения операции.

В случае access и split амортизированное время максимум $3 \log(s(t)/s(x)) + 1$, где x, t — корни дерева до и после операции, при чем $s(t) \leq W$, $s(x) \geq w(i)$ (если i в дереве) и $s(x) \geq \min\{w(i-), w(i+)\}$ (если i не в дереве). Если же элемента x нет в дереве, то есть либо $x-$, либо $x+$. Это дает нужную оценку на время access и split .

В случае join нам надо сделать только один access за время $3 \log(s(t_1)/w(i)) + 1$. Плюс к этому у нас увеличивается потенциал нашего дерева, сильно меньше чем на $3 \log(W/s(t_1))$

В insert мы делаем один access за время $3 \log \left(\frac{W - w(i)}{\min\{w(i-), w(i+)\}} \right) + 1$, а потенциал увеличивается максимум на $\log(W/w(i))$.

Оценка на delete берется из access и join . □